

UNIVERSIDADE FEDERAL DE RONDÔNIA
FACULDADE DE ENGENHARIA, TECNOLOGIA E INOVAÇÃO
ESTRUTURA DE DADOS I
LISTA PRÁTICA 1 – LISTAS

Listas estáticas e dinâmicas

Objetivo

O objetivo desta atividade é implementar o mesmo Tipo Abstrato de Dados (TAD) **Lista** utilizando três diferentes formas de representação:

1. **Lista Estática Sequencial;**
2. **Lista Estática Encadeada;**
3. **Lista Encadeada Dinâmica.**

A principal finalidade da atividade é observar que **o objeto manipulado e as operações disponibilizadas pelo TAD podem permanecer os mesmos, independentemente da estrutura de dados utilizada internamente para representá-lo.**

Assim, os três projetos deverão utilizar **o mesmo programa principal (main.c)**, sem qualquer alteração entre as três versões.

1. Organização da atividade

A atividade poderá ser realizada **em duplas**.

Cada dupla deverá desenvolver três projetos independentes:

- Projeto 1 – Lista Estática Sequencial;
- Projeto 2 – Lista Estática Encadeada;
- Projeto 3 – Lista Encadeada Dinâmica.

Os três projetos deverão apresentar o mesmo comportamento para o usuário.

A diferença deverá estar exclusivamente na **estrutura interna utilizada para armazenar os elementos e na implementação das operações.**

2. Tipo do elemento

Nas implementações realizadas em sala utilizamos inicialmente:

```
typedef struct{
    int chave;
} tipo_elem;
```

Nesta atividade, o objeto deverá possuir **ao menos mais um campo**, por exemplo:

```
typedef struct{
    int chave;
    char nome[50];
} tipo_elem;
```

A dupla poderá utilizar outro campo, caso deseje, desde que a estrutura possua obrigatoriamente:

- uma chave inteira, utilizada para pesquisa, ordenação e remoção;
- pelo menos **mais um atributo contendo informação associada ao elemento**.

Exemplos possíveis:

```
typedef struct{
    int chave;
    char nome[50];
    float valor;
} tipo_elem;
```

ou

```
typedef struct{
    int chave;
    char descricao[100];
} tipo_elem;
```

Atenção

A inclusão de novos campos em tipo_elem **não deve provocar alterações desnecessárias nas funções do TAD**.

Por exemplo, uma função que simplesmente remove um nó da lista não precisa conhecer individualmente nome, valor, descrição etc.

Ao realizar:

```
p->info = v;
```

todo o objeto `tipo_elem` é copiado.

Essa característica deverá ser observada durante a implementação.

3. Interface comum do TAD

Os três projetos deverão disponibilizar **as mesmas operações**, com as mesmas assinaturas.

As funções básicas estudadas em sala deverão ser mantidas:

```
void iniciar(Lista *L);

void destruir(Lista *L);

int vazia(Lista *L);

int inserir_inicio(Lista *L, tipo_elem v);

int inserir_final(Lista *L, tipo_elem v);

int inserir_ord(Lista *L, tipo_elem v);

int remover(Lista *L, int chave);

int remover_inicio(Lista *L);

int remover_final(Lista *L);

void exibir(Lista *L);

int buscar(Lista *L, int chave, tipo_elem *v);
```

Além dessas operações, deverão ser implementadas as funções adicionais descritas a seguir.

4. Novas operações

4.1 Tamanho da lista

Implementar uma função que retorne a quantidade de elementos existentes na lista.

```
int tamanho(Lista *L);
```

Exemplo:

Lista: 10 20 30 40

tamanho(&L)

Retorno: 4

4.2 Tamanho – versão recursiva

Implementar também uma versão recursiva da operação anterior.

```
int tamanho_rec(Lista *L);
```

A solução deverá efetivamente utilizar **recursividade**, e não apenas chamar a versão iterativa.

4.3 Busca recursiva

Além da função:

```
int buscar(Lista *L, int chave, tipo_elem *v);
```

deverá ser desenvolvida uma versão recursiva:

```
int buscar_rec(No *p, int chave, tipo_elem *v);
```

A função deverá retornar:

- 1, caso o elemento seja encontrado;
- 0, caso não seja encontrado.

Quando encontrado, o elemento completo deverá ser devolvido por meio do parâmetro v.

5. Exibição recursiva

Implementar uma segunda forma de exibição utilizando recursividade:

```
void exibir_rec(Lista *L);
```

A função deverá apresentar os elementos na mesma ordem lógica da lista.

Exemplo:

10 → 20 → 30 → 40

Saída:

10 20 30 40

6. Exibição em ordem inversa – versão recursiva

Implementar:

```
void exibir_inverso_rec(Lista *L);
```

A função deverá exibir os elementos da lista na ordem inversa **sem modificar a estrutura da lista**.

Exemplo:

Lista:

10 → 20 → 30 → 40

Saída:

40 30 20 10

Esta operação deverá obrigatoriamente utilizar **recursividade**.

7. Contagem condicional

Implementar uma função que informe quantos elementos possuem chave maior que determinado valor:

```
int contar_maiores(Lista *L, int chave);
```

Exemplo:

Lista:

10 25 30 42 50

Chamada:

```
contar_maiores(&L, 30);
```

Resultado:

2

pois 42 e 50 possuem chaves maiores que 30.

8. Contagem condicional recursiva

Desenvolver também a versão recursiva:

```
int contar_maiores_rec(Lista *L, int chave);
```

9. Requisito fundamental: o mesmo main.c

Este é um dos principais requisitos da atividade.

Os três projetos deverão utilizar **exatamente o mesmo algoritmo** main.c.

Portanto, chamadas como:

```
Lista L;
```

```
iniciar(&L);
```

```
inserir_inicio(&L, v);
```

```
inserir_final(&L, v);
```

```
inserir_ord(&L, v);
```

```
buscar(&L, chave, &v);
```

```
remover(&L, chave);
```

```
exibir(&L);
```

```
tamanho(&L);
```

```
destruir(&L);
```

deverão funcionar da mesma maneira nos três projetos.

Não será aceito

Criar, por exemplo:

```
inserir_estatica(...)
```

```
inserir_dinamica(...)
```

```
buscar_sequencial(...)
```

ou modificar o `main.c` de acordo com a estrutura utilizada.

O programa cliente **não deve saber qual estrutura de dados está sendo utilizada internamente.**

10. Projeto 1 – Lista Estática Sequencial

A primeira implementação deverá utilizar uma estrutura estática e sequencial.

Exemplo de organização:

```
#define MAX 100
```

```
typedef struct{  
    tipo_elem dados[MAX];  
    int n;  
} Lista;
```

A dupla deverá definir adequadamente a representação utilizada.

Deverão ser analisadas situações como:

- lista vazia;
- lista cheia;
- inserção no início;
- inserção no final;
- inserção ordenada;
- deslocamento de elementos;
- remoção;
- pesquisa;
- destruição lógica da lista.

11. Projeto 2 – Lista Estática Encadeada

Na segunda implementação, os elementos também deverão ser armazenados em uma área estática, porém o encadeamento deverá ser realizado por meio de **índices**.

Exemplo conceitual:

```
#define MAX 100
```

```
typedef struct{  
    tipo_elem info;  
    int prox;  
} No;
```

Uma possível estrutura para a lista seria:

```
typedef struct{  
    No nos[MAX];  
    int incio;  
    int dispo;  
} Lista;
```

A dupla deverá implementar uma estratégia para controlar:

- posição inicial da lista;
- próximo elemento;
- posições livres;
- reutilização de posições após remoções.

Não deverão ser utilizados `malloc()` e `free()` nesta implementação.

12. Projeto 3 – Lista Encadeada Dinâmica

A terceira implementação deverá utilizar alocação dinâmica de memória.

A estrutura será baseada no modelo estudado em sala:

```
typedef struct no{  
    tipo_elem info;  
    struct no *prox;  
} No;
```

```
typedef struct{
    No *head;
} Lista;
```

Nesta versão deverão ser utilizados corretamente:

`malloc()`

e

`free()`

Todo espaço alocado dinamicamente deverá ser liberado adequadamente.

13. Programa principal

O `main.c` deverá disponibilizar um menu que permita testar **todas as operações implementadas**.

Sugestão:

- 1 - Verificar se a lista está vazia
- 2 - Inserir no início
- 3 - Inserir no final
- 4 - Inserir ordenadamente
- 5 - Remover do início
- 6 - Remover do final
- 7 - Remover por chave
- 8 - Buscar elemento
- 9 - Exibir lista
- 10 - Exibir lista recursivamente
- 11 - Exibir lista em ordem inversa
- 12 - Informar tamanho
- 13 - Informar tamanho recursivamente
- 14 - Buscar recursivamente
- 15 - Contar chaves maiores que X
- 16 - Contar chaves maiores que X recursivamente
- 17 - Destruir lista
- 0 - Sair

A dupla poderá melhorar a organização e a apresentação do menu.

O importante é que **o mesmo arquivo `main.c` seja utilizado nos três projetos**.

14. Testes obrigatórios

Os programas deverão ser testados considerando, no mínimo:

- lista vazia;
- lista com um único elemento;
- lista com vários elementos;
- inserção no início;
- inserção no final;
- inserção em lista ordenada;
- remoção do primeiro elemento;
- remoção do último elemento;
- remoção de elemento intermediário;
- tentativa de remover uma chave inexistente;
- busca por elemento existente;
- busca por elemento inexistente;
- tentativa de inserção quando a lista está cheia, nas implementações estáticas;
- destruição da lista;
- reutilização da estrutura após sua destruição;
- execução das funções recursivas.

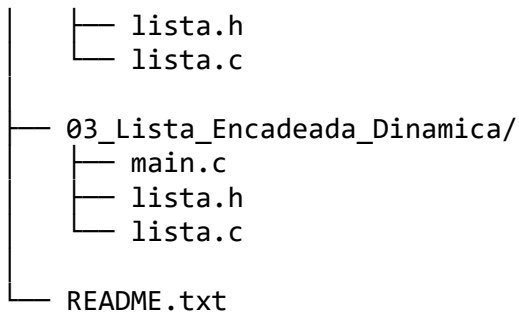
15. Organização dos arquivos

Cada implementação deverá estar em uma pasta separada.

A entrega deverá possuir uma organização semelhante a:

Lista_Prática_Dupla_Nome1_Nome2/

```
|
|— 01_Lista_Estatica_Sequencial/
|   |— main.c
|   |— lista.h
|   |— lista.c
|
|— 02_Lista_Estatica_Encadeada/
|   |— main.c
```



Os três arquivos `main.c` deverão ser **idênticos**.

Ao trocar de uma implementação para outra, o programa cliente deverá compilar utilizando apenas o respectivo `lista.h` e `lista.c`.

16. Arquivo README

Deverá ser incluído um arquivo:

`README.txt`

contendo:

- nome completo dos integrantes da dupla;
- turma;
- sistema operacional utilizado;
- compilador utilizado;
- versão do compilador, quando possível;
- IDE ou editor utilizado, se houver;
- instruções para compilação;
- eventuais observações necessárias para execução.

Exemplo:

Sistema operacional:

Windows 11

Compilador:

GCC 15.1.0 / MinGW-w64

IDE:

Code::Blocks

Compilação:

`gcc main.c lista.c -o lista`

As informações deverão corresponder ao ambiente realmente utilizado pela dupla.

17. Formato de entrega

Todos os projetos deverão ser reunidos em **um único arquivo .rar**.

Sugestão de nome:

Lista_Prática_Nome1_Nome2.rar

Não deverão ser entregues executáveis, arquivos temporários da IDE ou diretórios de compilação desnecessários.

Enviar apenas os arquivos necessários para compilar e executar os projetos.

18. Critérios de correção

A atividade será corrigida considerando não apenas se o programa executa, mas também a **qualidade da solução desenvolvida**.

Serão considerados:

Funcionalidade

- funcionamento correto das operações;
- tratamento adequado das situações de erro;
- funcionamento das três implementações;
- funcionamento correto das funções recursivas;
- manutenção do mesmo comportamento nos três projetos;
- ausência de acessos inválidos à memória.

Uso adequado das estruturas de dados

Será verificado se cada projeto realmente utiliza a representação solicitada:

- estática sequencial;
- estática encadeada;
- encadeada dinâmica.

Também será avaliado se a implementação respeita as características de cada estrutura.

Abstração do TAD

Será observado se:

- `main.c` é o mesmo nos três projetos;
- o programa cliente não manipula detalhes internos da estrutura;
- as operações apresentam as mesmas assinaturas;
- as diferenças de representação estão encapsuladas em `lista.h` e `lista.c`.

Boas práticas de programação

Serão considerados:

- organização do código;
- indentação;
- nomes de variáveis e funções;
- divisão adequada entre `.h`, `.c` e `main.c`;
- comentários quando necessários;
- ausência de duplicações desnecessárias;
- uso correto de constantes;
- inicialização adequada de variáveis;
- uso correto de ponteiros;
- uso correto de `malloc()` e `free()` na lista dinâmica;
- inexistência de vazamentos de memória.

Usabilidade

Também será avaliada a interação do usuário com o programa:

- menu compreensível;
- mensagens claras;
- solicitação adequada dos dados;
- apresentação legível dos elementos;
- mensagens para operações inválidas;
- tratamento de lista vazia e lista cheia.

Qualidade dos testes

O programa deverá demonstrar que diferentes situações foram consideradas e testadas.

19. Observação importante

O objetivo da atividade **não é escrever três programas completamente diferentes.**

O objetivo é perceber que podemos ter:

o mesmo objeto
+
as mesmas operações
+
o mesmo programa cliente

utilizando internamente:

representações diferentes

Essa é uma das ideias centrais do uso de **Tipos Abstratos de Dados.**

Ao final da atividade, deve ser possível substituir uma implementação da lista por outra sem modificar o algoritmo utilizado pelo programa principal.